

Xv6 文件系统架构

本指导书引用了 [rCore-Tutorial 指导书](#)的部分章节。

文件系统用途及接口

文件最早来自于计算机用户需要把数据持久保存在持久存储设备上的需求。由于放在内存中的数据在计算机关机或掉电后就会消失，所以应用程序要把内存中需要保存的数据放到持久存储设备的数据块(比如磁盘的扇区等)中存起来。随着操作系统功能的增强，在操作系统的管理下，应用程序不用理解持久存储设备的硬件细节，而只需对文件这种持久存储数据的抽象进行读写就可以了，由操作系统中的文件系统和存储设备驱动程序一起来完成繁琐的持久存储设备的管理与读写。所以本章要完成的操作系统的第一个核心目标是：让应用能够方便地把数据持久保存起来。

文件系统需要给用户程序提供根据文件路径查询文件的接口（open），还有根据文件描述符对查询到的文件进行文件操作的接口。

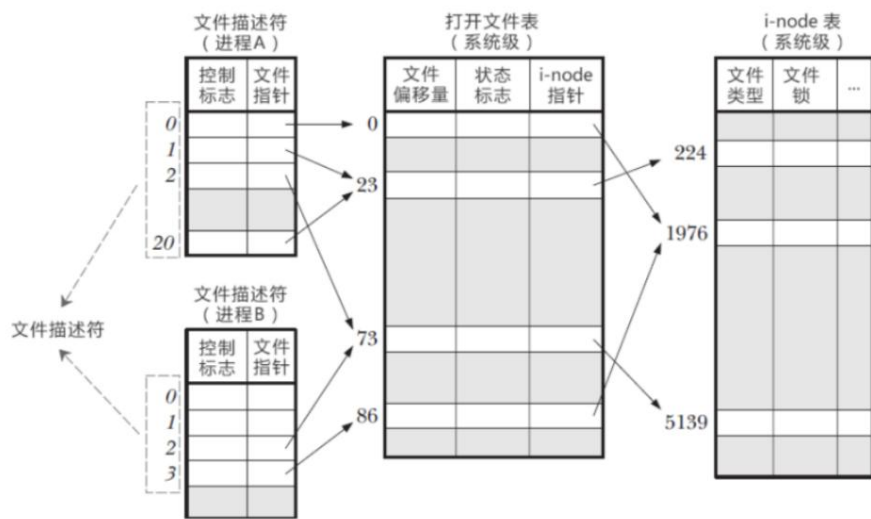
总体架构

Xv6 实现了分层的文件系统。从上到下依次分为名称层（像 `/usr/rtm/xv6/fs.c` 这样的路径，方便命名。），目录层（具有特殊内容的 inode，作为其他 inode 的列表），文件层（inode 分配器、读取、写入、元数据），日志层（用于多步骤更新的崩溃恢复，本实验基本不涉及到），块层（原始磁盘块的分配器）。

文件系统里的三个表

在 xv6 中，进程访问文件时会涉及三层表：首先，每个进程有自己的打开

文件表（文件描述符表），它把文件描述符映射到一个 struct file*；多个进程可以通过 fork 或 dup 指向同一个 struct file。其次，内核维护全局的**系统文件表**，其中的 struct file 记录了文件的读写权限、当前偏移量以及对 inode 的引用等运行时状态；当所有进程都关闭后，ref 计数归零，这个结构才会被回收。最后，struct file 往往指向一个 **inode 表项**（内存 inode cache 中的 struct inode），inode 代表文件的持久化元数据（大小、类型、磁盘块地址等），并且在全系统范围内唯一对应某个磁盘上的文件实体。三者关系可概括为：**fd → file → inode**，进程局部表面面向用户接口，全局文件表保存打开实例状态，而 inode 表则承载文件实体的核心信息。



```

// Per-process state
struct proc {
    struct spinlock lock;

    // p->lock must be held when using these:
    enum procstate state;           // Process state
    void *chan;                     // If non-zero, sleeping on chan
    int killed;                     // If non-zero, have been killed
    int xstate;                     // Exit status to be returned to parent's wait
    int pid;                         // Process ID

    // wait_lock must be held when using this:
    struct proc *parent;            // Parent process

    // these are private to the process, so p->lock need not be held.
    uint64 kstack;                  // Virtual address of kernel stack
    uint64 sz;                      // Size of process memory (bytes)
    pagetable_t pagetable;          // User page table
    struct trapframe *trapframe;    // data page for trampoline.S
    struct context context;          // swtch() here to run process
    struct file *ofile[NOFILE];     // Open files
    struct inode *cwd;               // Current directory
    char name[16];                  // Process name (debugging)
};

```

proc/proc.h 里 proc 结构体 ofile 为进程的打开文件表

```

struct {
    struct spinlock lock;
    struct file file[NFILE];
} ftable; //这是系统的全局打开文件表

```

fs/file.c 中系统的全局打开文件表

```

// 内核维护的 inode 表
struct
{
    struct spinlock lock;
    struct inode inode[NINODE];
} itable;

```

fs/fs.c 中内核维护的 inode 表

打开、关闭与读写文件的系统调用

文件打开

在读写一个常规文件之前，应用首先需要通过内核提供的 `sys_open` 系统调用让该文件在进程的文件描述符表（即上面提到的打开文件表）中占一项，并得到操作系统的返回值-文件描述符，即文件关联的表项在文件描述表中的索引值。文件描述符（File Descriptor, FD）是操作系统为进程访问已打开文件、设备或管道等 I/O 对象分配的一个非负整数索引，它存在于进程的打开文件表中，用来间接引用内核中的 `struct file` 结构。

```
/// 功能： 打开一个常规文件，并返回可以访问它的文件描述符。
/// 参数： path 描述要打开的文件的文件路径
/// omode 描述打开文件的标志。
uint64 sys_open(char* path,int omode)
```

进程的打开文件表

stdin	stdout	stderr		filea	
-------	--------	--------	--	-------	--

stdin,stdout,stderr,filea的文件描述符分别是0，1，2，4

文件描述符示例

文件关闭

在打开文件，对文件完成了读写操作后，还需要关闭文件，这样才让进程释放被这个文件占用的内核资源。`close` 的调用参数是文件描述符，当文件被关闭后，该文件在内核中的资源会被释放，文件描述符会被回收。这样，进程就不能继续使用该文件描述符进行文件读写了。

```
/// 功能： 当前进程关闭一个文件。
/// 参数： fd 表示要关闭的文件的文件描述符。
uint64 sys_close(int fd)
```

Xv6 中使用文件的流程为：

```
int fd = open(...)

read(fd,buf,5);

write(fd,buf,2);

close(fd);
```

文件的顺序读写

在打开一个文件之后，我们就可以用之前的 `sys_read/sys_write` 两个系统调用来对它进行读写了。需要注意的是，常规文件的读写模式和之前介绍过的几种文件有所不同。标准输入输出和匿名管道都属于一种流式读写，而常规文件则是顺序读写和随机读写的结合。由于常规文件可以看成一段字节序列，我们应该能够随意读写它的任一段区间的数据，即随机读写。然而用户仅仅通过 `sys_read/sys_write` 两个系统调用不能做到这一点。

事实上，进程为每个它打开的常规文件维护了一个偏移量，在刚打开时初始值一般为 0 字节。当 `sys_read/sys_write` 的时候，将会从文件字节序列偏移量的位置开始 **顺序** 把数据读到应用缓冲区/从应用缓冲区写入数据。操作完成之后，偏移量向后移动读取/写入的实际字节数。这意味着，下次 `sys_read/sys_write` 将会从刚刚读取/写入之后的位置继续。如果仅使用 `sys_read/sys_write` 的话，则只能从头到尾顺序对文件进行读写。当我们需要从头开始重新写入或读取的话，只能通过 `sys_close` 关闭并重新打开文件来将偏移量重置为 0。为了解决这种问题，有另一个系统调用 `sys_lseek` 可以调整进程打开的一个常规文件的偏移量，这样便能对文件进行随机读写。在 xv6 中并未实现这个系统调用，因为对于目前实验中的应用例子，顺序文件读写功能就已经足够满足需求了。顺带一提，在文件系统的底层实现中都是对文件进行随机读写的。

块层

实现磁盘块缓存功能的块缓存层的代码在 `fs/bio.c` 和 `fs/buf.h` 中。

由于操作系统频繁读写速度缓慢的磁盘块会极大降低系统性能，因此常见的手段是先通过 `virtio_disk_rw` 将一个块上的数据从磁盘读到内存中的一个缓冲区中，这个缓冲区中的内容是可以直接读写的，那么后续对这个数据块的大部分访问就可以在内存中完成了。如果缓冲区中的内容被修改了，那么后续还需要通过 `virtio_disk_rw` 将缓冲区中的内容写回到磁盘块中。

事实上，无论站在代码实现鲁棒性还是性能的角度，将这些缓冲区合理的管理起来都是很有必要的。一种完全不进行任何管理的模式可能是：每当要对一个磁盘块进行读写的时候，都通过 `virtio_disk_rw` 将块数据读取到一个 *临时* 创建的缓冲区，并在进行一些操作之后（可选地）将缓冲区的内容写回到磁盘块。从性能上考虑，我们需要尽可能降低实际块读写（即 `virtio_disk_rw`）的次数，因为每一次调用它们都会产生大量开销。要做到这一点，关键就在于对块读写操作进行 **合并**。例如，如果一个块已经被读入缓冲区中了，那么我们就没有必要再读一遍，直接用已有的缓冲区就行了；同时，对于缓冲区中的同一个块的多次修改没有必要每次都写回磁盘，只需等所有的修改都结束之后统一写回磁盘即可。当磁盘上的数据结构比较复杂的时候，很难通过应用来合理地规划块读取/写入的时机。这不仅可能涉及到复杂的参数传递，稍有不慎还有可能引入同步性问题（目前可以暂时忽略）：即一个块缓冲区修改后的内容在后续的同个块读操作中不可见，这很致命但又难以调试。

因此，xv6 的做法是将缓冲区统一管理起来。当我们要读写一个块的时候，首先就是去全局管理器中查看这个块是否已被缓存到内存缓冲区中。如果是这样，则在一段连续时间内对于一个块进行的所有操作均是在同一个固定的缓冲区中进行的，这解决了同步性问题。此外，通过 `virtio_disk_rw` 进行块实际读写的时机完全交给块缓存层的全局管理器处理，上层子系统无需操心。全局管理器会

尽可能将更多的块操作合并起来，并在必要的时机发起真正的块实际读写。

块层给上层提供的接口：

Bread:返回一个已上锁且从磁盘中读取了数据的缓存块。

Bwrite:将指定缓存块写回磁盘。（只被 log 层使用，更上层的写回通过调用 log 层实现）

Brelease:释放一个缓存块。

Bpin/bunpin:修改缓存块的引用计数，只被 log 层使用。

块缓存使用的 **blockno** 与使用块缓存的流程

在 xv6 里，bread 的参数 blockno 指的是磁盘上全局的块编号，也叫物理块号，也就是相对于整个文件系统映像的第几个块（从 0 开始计数）。文件系统把磁盘划分为若干固定大小的块，每个块都用一个唯一的编号标识。比如：块号 1 存放超级块（superblock），接下来的若干块是 inode 表，再往后是位图块和数据块。bread 通过这个 blockno 找到对应的磁盘块，把它读入 buffer cache，并返回一个 struct buf 供后续修改或读取。

Xv6 使用了 log_write 来代替 bwrite，log_write 起到了 bwrite 的作用（具体原理可以不用深究）。**Xv6 中使用块缓存的流程为：**

bp = bread(...)

modify bp->data[]

log_write(bp)

brelse(bp)

这段流程展示了在 xv6 中对磁盘块进行安全修改的标准步骤。首先通过 bread 把目标磁盘块读入缓存，得到一个 struct buf；然后在内存中对 bp->data[]

的内容进行修改；接着调用 `log_write`，而不是直接 `bwrite`，把该缓冲区加入到日志系统的事务中，确保修改会以原子方式写入磁盘，避免系统崩溃时文件系统出现不一致；最后调用 `brelease` 释放缓存块，使它重新可供其他使用者访问。整个过程保证了既能修改磁盘数据，又能保持文件系统的可靠性。Xv6 每次在内存中修改完目标磁盘块的数据后都需要进行写回再释放缓存块，来保证修改能被写回磁盘。

Xv6 磁盘布局

在 xv6 磁盘布局中，按照块编号从小到大顺序地分成 6 个不同属性的连续区域：最开始的区域长度为一个块，为引导块，保留给引导程序使用。使用 riscv 架构的 xv6 不会用到引导块和引导程序，内核是直接被加载到 qemu 中运行。

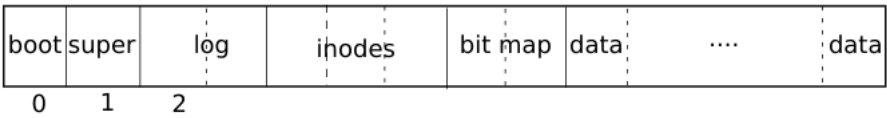
第二个区域长度是一个块，为 xv6 文件系统 **超级块** (Super Block)。超级块内以魔数的形式提供了文件系统合法性检查功能，同时还可以定位其他连续区域的位置。

第三个区域是日志区域，长度为若干个块，用来存放文件系统的日志记录，确保崩溃恢复时一致性。

第四个区域是索引节点区域，长度为若干个块。其中的每个块都存储了若干个索引节点。

第五个区域是一个数据块位图，长度为若干个块。它记录了后面的数据块区域中有哪些数据块已经被分配出去使用了，而哪些还尚未被分配出去。

最后的区域则是数据块区域，顾名思义，其中的每一个已经分配出去的块保存了文件或目录中的具体数据内容。



Xv6 磁盘布局图

Superblock 操作—数据块，inode 的分配和释放

实现 superblock 操作的代码在 `fs/fs.c` 中。

新建文件时，我们需要从磁盘中分配一个空闲的 inode，当文件需要扩充大小时需要为它分配空闲数据块。当文件被删除时，我们需要回收它的 inode 和它被分配的数据块。Xv6 使用 `ialloc` 函数分配空闲 inode，使用 `balloc` 函数分配一个空闲数据块，使用 `bfree` 函数释放一个数据块，使用 `iput` 函数释放一个 inode。Superblock 结构中存在磁盘上 inode 区，数据块区的起始块号和区域长度，还有数据块 bitmap 区的起始块号。我们可以根据这些数据定位到磁盘中所有的 inode 和数据块。

Superblock 提供的接口：

Ialloc:从磁盘中分配一个空闲 inode。

Iput:将 inode 缓存的引用计数减少一，当引用计数为 0 时 inode 缓存会被释放。

在 inode 的 link 数为 0 时，还会将这个 inode 释放给磁盘。

Balloc:从磁盘中分配一个空闲数据块。

Bfree:释放一个数据块给磁盘。

Superblock 的初始化

Xv6 声明了一个全局 Superblock 结构 `sb`，在文件系统初始化时 `readsb` 函数会将磁盘中的 superblock 读到 `sb` 中，之后所有关于 superblock 的操作都是通过内存中的 `sb` 完成。

文件层（或者叫 inode 层）

一个 inode 描述了一个没有名字的文件。inode 磁盘结构保存了元数据：文件的类型、文件大小、引用该文件的链接数，以及存储文件内容的块列表。

inode 按顺序排列在磁盘上的 sb.inodestart 位置，存储在 sb 中。每个 inode 都有一个编号，表示它在磁盘上的位置，即它是从前往后数第几个 inode(从 1 开始)。给定一个 inum，可以通过公式算出它在磁盘的具体位置。

内核在内存中保持一个正在使用的 inode 表，以便同步多个进程访问共享 inode。内存中的 inode 包含一些不在磁盘上存储的管理信息：ip->ref 和 ip->valid。inode 及其内存表示会经过一系列的状态转换，才能被文件系统的其他代码使用。

Inode 层提供给上层的接口：

对 inode 进行管理的接口：

iget:根据 inode 编号在 inode 表中申请一个位置，此时 inode 还未从磁盘中读取。

ilock:将 inode 表中指定 inode 从磁盘读取进来并上锁，便于后续操作。

iunlock: 将 inode 表中指定 inode 解锁。

iupdate:将指定 inode 同步回磁盘。

iput:将 inode 缓存的引用计数减少一，当引用计数为 0 时 inode 缓存会被释放。

在 inode 的 link 数为 0 时，还会将这个 inode 释放给磁盘。

idup:将 inode 缓存的引用计数增加一。

对 inode 里存的数据进行管理的接口：

Bmap:将文件的逻辑块号转成磁盘上的物理块号（详细解释在下面）。

Itrunc:将 inode 清空。

Stati:把 inode 状态拷贝进传入参数指定地址处。

Readi:将文件指定偏移处数据复制到指定缓冲区中。

Writei: 将指定缓冲区中数据复制到文件指定偏移处,必要时会增长文件长度。

Inode 层使用流程

* 分配：当 inode 的类型（磁盘上的）非零时， inode 就会被分配。ialloc() 分配 inode， 当引用计数和链接计数降为零时， iput() 释放 inode。

* 表中引用：inode 表中的一个条目如果 ip->ref 为零， 就是空闲的。否则， ip->ref 跟踪指向该条目的 内存指针数量（打开的文件和当前目录）。iget() 查找或创建一个表条目并递增其引用计数；iput() 则递减引用计数。

* 有效：inode 表条目中的信息（类型、大小等） 只有在 ip->valid 为 1 时才是正确的。 ilock() 从磁盘读取 inode 并设置 ip->valid， 而 iput() 在 ip->ref 降为零时清除 ip->valid。

* 锁定：文件系统代码只有在首先锁定 inode 后， 才能检查和修改 inode 及其内容中的信息。

因此，一个典型的操作序列是：

ip = iget(dev, inum)

ilock(ip)

... 检查和修改 ip->xxx ... （修改后需要先将 inode 写回磁盘再释放缓存）

iunlock(ip)

iput(ip)

ilock() 与 iget() 分开，以便系统调用可以 获取一个长时间的 inode 引用（例如打开的文件），并且只在短时间内锁定它（例如在 read() 中）。这种分离还有助于避免死锁和路径查找中的竞争条件。iget() 递增 ip->ref，确保 inode 保持在表中，并且指向它的指针保持有效。许多内部文件系统函数要求调用者在调用时已经锁定了相关 inode；这使得调用者可以创建多步骤的原子操作。

itable.lock 自旋锁保护 itable 条目的分配。由于 ip->ref 表示一个条目是否空闲，而 ip->dev 和 ip->inum 表示条目所持有的 inode，因此在使用这些字段时，必须持有 itable.lock。

ip->lock 睡眠锁保护除 ref、dev 和 inum 外的所有 ip-> 字段。必须持有 ip->lock 才能读取或写入 inode 的 ip->valid、ip->size、ip->type 等字段。

文件的逻辑块号和物理块号

文件的逻辑块号指的是文件内部相对于文件开头的块序号，从 0 开始计数。比如文件的第一个 1KB 数据对应逻辑块号 0，第二个 1KB 对应逻辑块号 1，以此类推。逻辑块号只反映文件的内部结构，不关心这些数据实际存储在磁盘的哪个位置。

物理块号指的是磁盘上全局的块编号，也就是块层使用的块编号，从整个磁盘的起始块开始顺序编号。它表示数据真正存放在哪个磁盘块上，是文件系统通过分配、管理磁盘空间得到的结果。

bmap 函数就是负责把某个文件的**逻辑块号**转换成对应的**物理块号**，这样文件系统就能根据文件的偏移正确找到存储在磁盘上的数据。

目录层和名称层

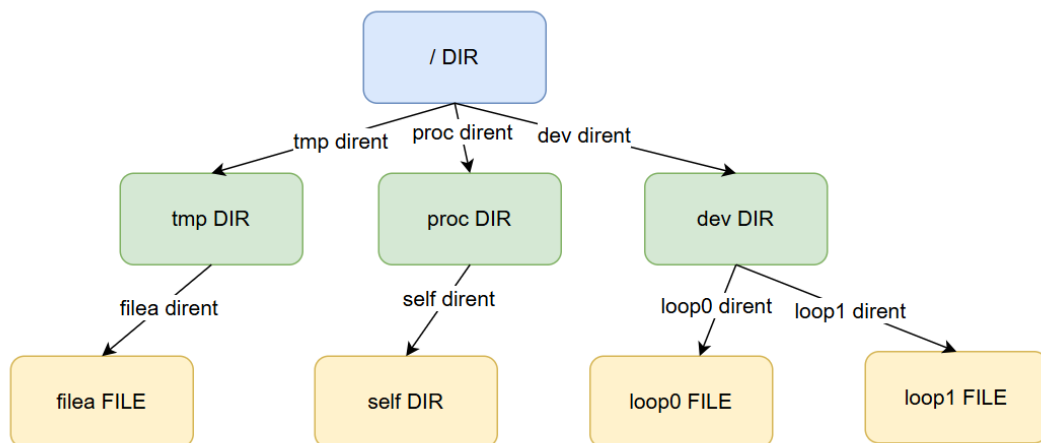
目录和目录项

目录、目录项相关操作和解析路径相关代码在 fs/fs.c 中

在 xv6 中，目录（directory）本质上就是一种特殊的文件，它的 inode 类型是 T_DIR。和普通文件不同，目录的内容不是用户数据，而是由一系列目录项（directory entry）组成。每个目录项用 struct dirent 表示，包含两个字段：inum 和 name。一个目录项就是“文件名 → inode 编号”的映射。通过 inode 编号，文件系统可以找到目标文件或子目录的元信息 and 数据块。目录文件的内容就是这些 dirent 顺序排列的数组，因此可以像普通文件那样用 readi、writei 来读写目录项。查找文件时，xv6 会扫描目录文件，逐项比对文件名并找到它的 inode 号，然后我们就可以根据这个 inode 号找到这个文件的 inode（具体怎么找可以看上面的文件层/inode 层）。

文件系统通过目录将所有文件组织成一棵树：树根是根目录 /，它的 inode 编号固定为 1；目录作为一种特殊文件，其内容是由一系列目录项组成，每个目录项保存文件名和对应的 inode 编号，从而把名字和具体的文件或子目录关联起来；目录项中的 “.” 指向自身，“..” 指向父目录，使得目录间能形成层次结构；路径解析时，内核会从根目录开始，逐级扫描目录项找到目标 inode。借助这种机制，普通文件、子目录和设备文件都被挂接到目录树中，整个文件系统便形成了一个统一的树状结构。

目录文件起到了类似“邻接表”的作用，它记录了“当前目录节点”能直接到达哪些文件或子目录。每个目录文件里的一条目录项（dirent），就像邻接表里的一条边，保存了“文件名 → inode 编号”的映射。通过 inode 编号，系统就能找到目标文件或子目录的详细信息。



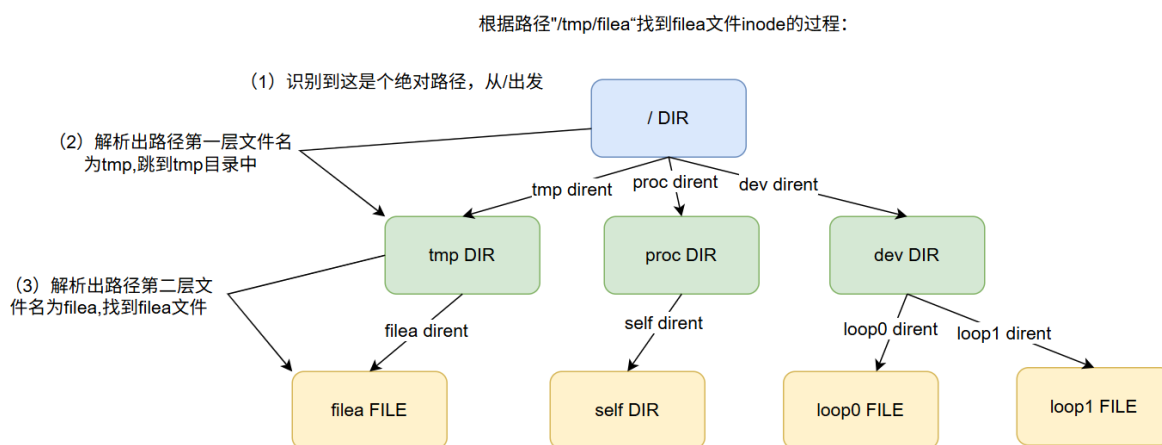
文件形成的树状结构示例

open、link、unlink 等系统调用会把他们指定的文件的路径作为系统调用参数传到内核中。文件路径（file path）是用来标识文件在目录树中位置的字符串，它描述了从根目录或当前目录到目标文件或子目录的访问路线。路径可以分为两类：

绝对路径以根目录 / 开头，例如 /usr/README，表示从根目录开始逐级访问目录项找到目标文件。

相对路径不以 / 开头，例如 docs/file.txt，表示从当前工作目录（即 proc 结构里的 cwd 字段）出发，按照路径中各个目录名逐级查找目标文件。

在路径解析过程中，xv6 内核会从起始目录读取目录文件，逐条扫描目录项，比对每一级目录名，并通过 inode 编号找到下一级目录或文件。这个过程就是在文件系统的目录树上一层一层地进行遍历，根据路径中每一层解析出的文件名决定接下来跳到当前目录在文件树上的哪一个孩子文件中。



解析路径流程

目录层提供给上层的接口：

Dirlookup:在文件夹中查询指定文件名的文件，返回这个文件的 **inode**，还有这个 **inode** 目录项在文件夹中的偏移量。

Dirlink:指定文件夹中创建指定文件名的目录项。

Namei/nameiparent:根据路径查询这个路径对应的文件，返回文件的 **inode**。

File 层

在实现了下面层那些接口之后，file 层的操作就是使用下面层的接口完成对应的file 操作了，这部分代码较容易理解，直接阅读 xv6 源码也很容易理解。难的其实是如何实现下面层。